

コンパイラ (5)

2026/10/16

LALR 構文解析

LR(1) の近似: 状態数は SLR(1) と同じくらいにする

LR(0) 項が同じの LR(1) 項は同じ状態を表す

⇒ 同じ状態にまとめる

G_{10} の場合: I_3 と I_{13} 、 I_6 と I_{10} 、 I_8 と I_{11} 、 I_9 と I_{12} を統合

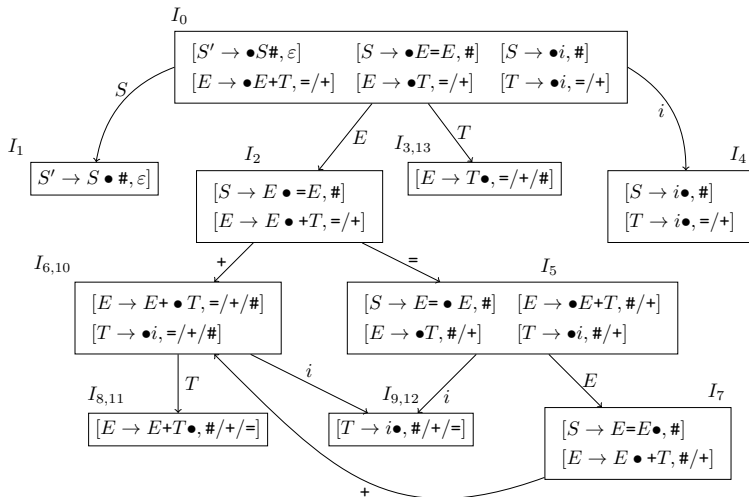
これで conflict が発生しなければ LALR(1) 構文解析可能

yacc(bison), sableCC の構文解析法

LR 遷移系は SLR と同等

還元するときは LR(1) 項の先読み情報を使う = 文脈に依存した還元情報

$G_{=}$ の LALR(1) 遷移系

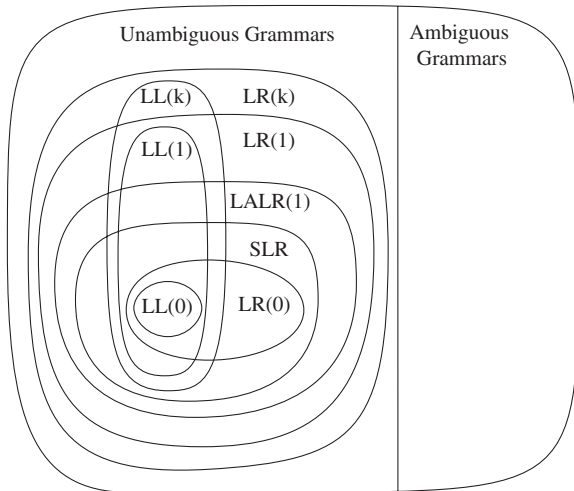


$G_{=}$ の LALR(1) 構文解析表

	action				goto		
	i	=	+	#	S	E	T
0	s4				1	2	(3,13)
1				acc			
2		s5	s(6,10)				
(3,13)		r4	r4	r4			
4		r5	r5	r2			
5	s(9,12)					7	(3,13)
(6,10)	s(9,12)						(8,11)
7			s(6,10)	r1			
(8,11)		r3	r3	r3			
(9,12)		r5	r5	r5			

$G_{=}$ は LALR(1) 構文解析可能である。

構文のクラスわけ



(“Tiger book” より)

LL(1) と LALR(1)

LL(1) ではあるが LALR(1) でない文法

$$\left\{ \begin{array}{ll} 1: S \rightarrow (X & 5: X \rightarrow F] \\ 2: S \rightarrow E] & 6: E \rightarrow A \\ 3: S \rightarrow F) & 7: F \rightarrow A \\ 4: X \rightarrow E) & 8: A \rightarrow \end{array} \right\}$$

LALR(1) であるが LL(1) でない文法

$$\left\{ \begin{array}{ll} 1: S \rightarrow Aa & 4: S \rightarrow bBa \\ 2: S \rightarrow bAc & 5: A \rightarrow d \\ 3: S \rightarrow Bc & 6: B \rightarrow d \end{array} \right\}$$

演算子順位法

$$G_{Exp} = \langle \{E\}, \{+, -, *, /, (,), i, n\}, P, E \rangle$$

$$P = \left\{ \begin{array}{ll} E \rightarrow E + E, & E \rightarrow E - E, \\ E \rightarrow E * E, & E \rightarrow E / E, \\ E \rightarrow (E), & E \rightarrow i, \\ E \rightarrow n \end{array} \right\}$$

	i	n	$+$	$-$	$*$	$/$	$\#$
i			$\gg$	$\gg$	$\gg$	$\gg$	$\gg$
n			$\gg$	$\gg$	$\gg$	$\gg$	$\gg$
$+$	$\ll$	$\ll$	$\gg$	$\gg$	$\ll$	$\ll$	$\gg$
$-$	$\ll$	$\ll$	$\gg$	$\gg$	$\ll$	$\ll$	$\gg$
$*$	$\ll$	$\ll$	$\gg$	$\gg$	$\gg$	$\gg$	$\gg$
$/$	$\ll$	$\ll$	$\gg$	$\gg$	$\gg$	$\gg$	$\gg$
$\#$	$\ll$	$\ll$	$\ll$	$\ll$	$\ll$	$\ll$	acc

$op_1 \ll op_2$: op_2 を op_1 よりも先に還元する
一番優先順位の高い演算子から還元する

演算子順位構文表

	i, n	$+, -$	$*, /$	$($	$)$	$\#$
i, n		$\gg$	$\gg$		$\gg$	$\gg$
$+, -$	$\ll$	$\gg$	$\ll$	$\ll$	$\gg$	$\gg$
$*, /$	$\ll$	$\gg$	$\gg$	$\ll$	$\gg$	$\gg$
$($	$\ll$	$\ll$	$\ll$	$\ll$	$==$	
$)$		$\gg$	$\gg$		$\gg$	$\gg$
$\#$	$\ll$	$\ll$	$\ll$	$\ll$	$\ll$	<i>acc</i>

ハンドラ： $\ll$ と $\gg$ で囲まれた部分
還元して非終端記号に置き換える

例 4.5

 $\#i + i * i\#$ $\#\underline{\ll i \gg} + \ll i \gg * \ll i \gg \#$ $i + i * i$

例 4.5

 $\#E + i * i\#$ $\# \ll + \ll \underline{i} \gg * \ll i \gg \#$ E

|

 $i + i * i$

例 4.5

 $\#E + E * i\#$ $\# \ll + \ll * \ll \underline{i} \gg \#$
$$\begin{array}{ccccccc} E & & E & & & & \\ | & & | & & & & \\ i & + & i & * & i & & \end{array}$$

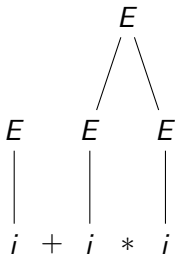
例 4.5

$$\#E + E * E\# \quad \# \ll + \ll * \gg \#$$
$$\begin{array}{c} E \\ | \\ i \end{array} + \begin{array}{c} E \\ | \\ i \end{array} * \begin{array}{c} E \\ | \\ i \end{array}$$

例 4.5

$\#E + E\#$

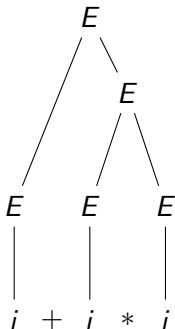
$\# \ll + \gg \#$



例 4.5

#E#

##



例

 $\#(i + i) * i\#$ $\# \ll (\ll \underline{i} \gg + \ll i \gg) \gg * \ll i \gg \#$ $(i + i) * i$

例

 $\#(E + i) * i\#$ $\# \ll (\ll + \ll \underline{i} \gg) \gg * \ll i \gg \#$
$$\begin{array}{c} E \\ | \\ (i + i) * i \end{array}$$

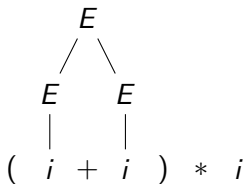
例

$$\#(E + E) * i\#$$

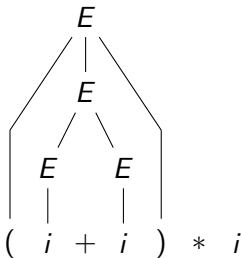
$$\# \ll (\ll + \gg) \gg * \ll i \gg \#$$

$$\begin{array}{c} E \quad E \\ | \quad | \\ (\quad i + i \quad) * i \end{array}$$

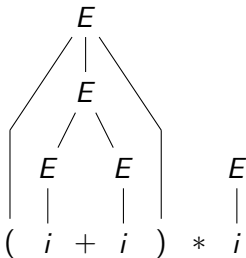
例

 $\#(E) * i \#$ $\# \ll (==) \gg * \ll i \gg \#$ 

例

 $\#E * i\#$ $\# \ll * \ll \underline{i} \gg \#$ 

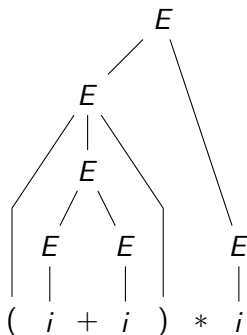
例

 $\#E * E\#$ $\# \llcorner * \lrcorner \#$ 

例

#E#

##



構文エラー表示と誤り回復

$w \notin L(G)$ の場合は実行する必要はない。

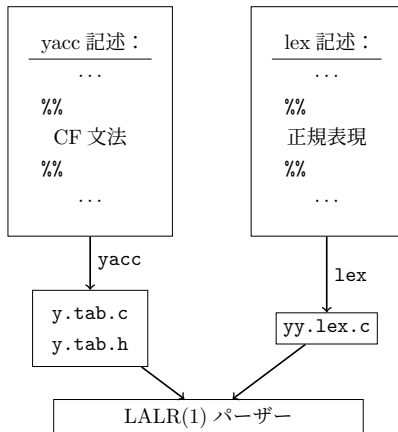
構文解析時に構文解析表に空欄に至った場合エラーが発生。
ユーザーに対してエラーの原因を提示して、修正を促す。

- ▶ 正しい構文解析が可能な修正を最小限行う。
文法が限定される。修正が限定的でないと難しい。(スコアで提示するなどの方法はコストが大きいため、あまり用いられない)
- ▶ パニックモードに入り、特定の分離記号 (デリミター) まで読みとばす。} や; など。

回復することによって複数のエラーを指摘することが可能になる。2つめ以降は回復に依存

yacc/lex を用いた構文解析

- ▶ yacc(yet another compiler compiler) (または bison)
CF 文法から構文解析を実行する PDT を自動生成.



例：四則演算 (calc.y)

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex(void);
void yyerror(const char *s);
%}

/* トークン定義 */
%token NUMBER

/* 演算子の優先順位と結合性 */
%left '+' '-'
%left '*' '/'
%nonassoc UMINUS

%%

/* 構文規則 */
input:
/* 空入力を許容 */
| input line
;
```

```
line:
'\n'      { printf("> "); }
| expr '\n' { printf("= %d\n", $1); }
| error '\n' { yyerror; printf("> "); }
;

expr:
NUMBER    { $$ = $1; }
| expr '+' expr { $$ = $1 + $3; }
| expr '-' expr { $$ = $1 - $3; }
| expr '*' expr { $$ = $1 * $3; }
| expr '/' expr {
    if ($3 == 0) {
        yyerror("division by zero");
        $$ = 0;
    } else {
        $$ = $1 / $3;
    }
}
| '-' expr %prec UMINUS { $$ = -$2; }
| '(' expr ')' { $$ = $2; }
;

%%

void yyerror(const char *s) {
    fprintf(stderr, "Error: %s\n", s);
}

int main(void) {
    setvbuf(stdout, NULL, _IONBF, 0);

    printf("電卓プログラム (Ctrl+D または Ctrl+C で終了)\n> ");
    return yyparse();
}
```

例：四則演算 (calc.l)

```
%{
#include <stdio.h>
#include <stdlib.h>

/* y.tab.h より前に YYSTYPE を定義、または extern 宣言 */
#ifndef YYSTYPE
typedef int YYSTYPE;
#endif
extern YYSTYPE yylval;

#include "y.tab.h" /* yacc が生成するトークン定義を取り込む */
}%

%%

[0-9]+    {
            yyval = atoi(yytext);
            return NUMBER;
        }

[ \t]      ; /* 空白・タブは無視 */

\n         { return '\n'; }

.          { return yytext[0]; }

%%

int yywrap(void) {
    return 1;
}
```

```
yacc -d calc.y
lex calc.l
cc y.tab.c lex.yy.c \
-o calc
```

例：四則演算 (calc.py)

```
from sly import Lexer, Parser
```

```
class CalcLexer(Lexer):
```

```
    # トークン名のセット
```

```
    tokens = {NUMBER}
```

```
    # 1 文字の演算子・記号（そのままリテラルとして扱う）
```

```
    literals = {'+', '-', '*', '/', '(', ')', '\n'}
```

```
    # 無視する文字（空白とタブ）
```

```
    ignore = ' \t'
```

```
    # 数値のトークン化規則（整数と小数の両方に対応）
```

```
    @_(r'\d+(\.\d+)?')
```

```
    def NUMBER(self, t):
```

```
        t.value = float(t.value) if '.' in t.value else int(t.value)
```

```
        return t
```

```
    # 改行処理
```

```
    @_(r'\n+')
```

```
    def ignore_newline(self, t):
```

```
        self.lineno += t.value.count('\n')
```

```
    # 字句エラー処理
```

```
    def error(self, t):
```

```
        print(f"不正な文字です: '{t.value[0]}' (行 {self.lineno})")
```

```
        self.index += 1
```

```
class CalcParser(Parser):
```

```
    # 利用するトークン定義を Lexer から受け取る
```

```
    tokens = CalcLexer.tokens
```

```
    # 演算子の優先順位と結合性（下ほど優先順位が高い）
```

```
    precedence = (
```

```
        ('left', '+', '-'),
```

```
        ('left', '*', '/'),
```

```
        ('right', 'UMINUS'), # 単項マイナス
```

```
)
```

```
    # 文法規則: expr の計算
```

```
    @_('expr "+" expr')
```

```
    def expr(self, p):
```

```
        return p.expr0 + p.expr1
```

```
    @_('expr "-" expr')
```

```
    def expr(self, p):
```

```
        return p.expr0 - p.expr1
```

```
    @_('expr "*" expr')
```

```
    def expr(self, p):
```

```
        return p.expr0 * p.expr1
```

```
    @_('expr "/" expr')
```

```
    def expr(self, p):
```

```
        if p.expr1 == 0:
```

```
            print("Error: division by zero")
```

```
            return 0
```

例：四則演算 (calc.py)

```
# 単項マイナス (%prec UMINUS に相当)
```

```
@_('-' expr %prec UMINUS')
```

```
def expr(self, p):
```

```
    return -p.expr
```

```
# 括弧
```

```
@_('(' expr ')')
```

```
def expr(self, p):
```

```
    return p.expr
```

```
# 数値
```

```
@_('NUMBER')
```

```
def expr(self, p):
```

```
    return p.NUMBER
```

```
# 構文エラー処理
```

```
def error(self, p):
```

```
    if p:
```

```
        print(f"構文エラー: '{p.value}' (位置: {p.index})")
```

```
    else:
```

```
        print("構文エラー: 式が途中で終わっています")
```

```
if __name__ == '__main__':
```

```
    lexer = CalcLexer()
```

```
    parser = CalcParser()
```

```
print("SLY 電卓プログラム
```

```
(Ctrl+C または Ctrl+D で終了)")
```

```
while True:
```

```
    try:
```

```
        text = input('> ')
```

```
    except (EOFError, KeyboardInterrupt):
```

```
        print("\n 終了します。")
```

```
        break
```

```
    if text.strip():
```

```
        result =
```

```
            parser.parse(lexer.tokenize(text))
```

```
    if result is not None:
```

```
        print(f"= {result}")
```